



Errata Version 1.0

EZ-Host ä

CY7C67300

Date Code 0316 and Later

Silicon Revision A1

1. UART Does Not Recognize Framing Errors

Description: The UART is not designed to recognize framing errors.

Implications: On some platforms this can cause EZ-Host to see a string of NULL characters and cause the UART to get out of sync.

This can cause the UART to lose connection with the host during serial communicating. One example of this is if the UART is used as the debug port to the PC. This problem has occurred on, but is not limited to, Dell machines running WinXP or Win2K.

Workaround: For general use, there is no workaround.

If this problem is experienced while debugging, try running the debugger on a different Host (PC/OS). Otherwise the USB port can be used for the debugging interface.

Status: No fix is currently planned for future silicon versions.

2. UART Does Not Override GPIO Control Register

Description: When the UART is enabled, the GPIO Control Register still has control over GPIO 6 (UART RX pin). When enabled, the UART should override the GPIO Control Register, which defaults to setting the pin as an output.

Implications: If the GPIO Control Register is written to set GPIO 6 as an output, then data will not be received on the UART interface.

Workaround: Make sure the GPIO Control Register is written appropriately to set GPIO6 as an input when the UART is enabled.

Status: No fix is currently planned for future silicon versions.

3. VBUS Interrupt (VBUS Valid) Requires Debouncing

Description: The VBUS interrupt in the Host/Device Status Registers [0xC090 and 0xC0B0] and OTG Control Register [0xC098] triggers multiple times whenever VBUS is turned on. It should only trigger once when VBUS rises above 4.4V and once when VBUS falls from above 4.4V to 0V.

Implications: If enabled, multiple VBUS Interrupts may occur when VBUS rises.

Workaround: When reading the status of this interrupt, a software debounce should be implemented.

Status: No fix is currently planned for future silicon versions.

4. Coupled SIE Interrupt Enable Bits

Description: Some Host/Device 1 SIE events will still trigger an interrupt when only the Host/Device 2 SIE Interrupt Enable is set and vice versa.

Implications: The Host/Device global Interrupt Enable bits can not be used to disable each Host/Device SIE independently. These bits are found in the Interrupt Enable Register (0xC00E).

Workaround: If an SIE Interrupt is desired, both Host/Device 1 and Host/Device 2 Interrupt Enable bits should be set in the global Interrupt Enable Register (0xC00E).

To properly mask an SIE Interrupt to a single SIE, the lower level Host/Device Interrupt Enable Registers (0xC08C and 0xC0AC) must be used. For example, setting the Host/Device 2 IE Register to 0x0000 will prevent any Host/Device 2 events from generating a Host/Device interrupt.

To disable all SIE interrupts, both Host/Device Interrupt Enable bits in the Interrupt Enable Register should be cleared.

Status: No fix is currently planned for future silicon versions.

5. Un-Initialized SIExmsg Registers

Description: The SIE1msg and SIE2msg Registers [0x0144 and 0x0148] are not initialized at power up.

Implications: If using the HPI interface in coprocessor mode, random data will be written to the SIE1msg and SIE2msg Registers [0x0144 and 0x0148] at power up. This will cause two improper HPI interrupts (HPI_INTR) to occur, one for each of the two SIExmsg Registers.

Workaround: The external processor should clear the SIExmsg Registers [0x0144 and 0x0148] shortly after nRESET is de-asserted and prior to the expected processing of proper HPI interrupts (generally 10ms after nRESET is de-asserted).

Status: No fix is currently planned for future silicon versions.

6. BIOS USB Peripheral Mode: Descriptor Length

Description: The BIOS will not properly return a descriptor or set of descriptors if the length is a multiple of the control endpoint's maximum packet size.

Implications: If the descriptor length is a multiple of the maximum packet size, then the BIOS will respond with a STALL instead of a zero-length data packet for the final IN request.

Workaround: If the requested descriptor length is a multiple of the maximum packet size, then either the maximum packet size or the descriptor length needs to change.
A descriptor length can be increased by simply adding a padded byte to the end of a descriptor and increasing the descriptor Length byte by one. Section 9.5 (Descriptor) of the USB2.0 specification allows a descriptor length to be larger than the value defined in the specification.

Status: No fix is currently planned for future silicon versions.

Revision History

Revision	Date Issue	Change
1.0	May 12, 2003	Creation